

docs.google.com – 全画面表示を終了するには、Esc を押します

TL1

01_10.Boxコライダー

日本工学院専門学校
デザインカレッジ

まとめ

日本工学院専門学校
デザインカレッジ

今回、新しいことは出てこなかったが

- カスタムプロパティの追加（オペレータ）
- カスタムプロパティの設定（パネル）
- カスタムプロパティの描画反映（描画関数）

を連動させることで、コライダー周りの機能を追加した。

既に一通りの機能を扱ってきたので、さらに自分で必要な機能を拡張して実用的なレベルエディタを作ろう。

また、この技術はモデル単体のエクスポーターにも使えるので、独自形式のモデル出力などにも挑戦してみるとよいだろう。

ファイル出力に反映する

日本工学院専門学校
デザインカレッジ

level_editor.py -class MYADDON_OT_export_scene

```
#カスタムプロパティ'file_name'
if "file_name" in object:
    self.write_and_print(file, indent + "N %s" % object["file_name"])
#カスタムプロパティ'collision'
if "collider" in object:
    self.write_and_print(file, indent + "C %s" % object["collider"])
    temp_str = indent + "CC %f %f %f"
    temp_str %= (object["collider_center"][0],object["collider_center"][1],object["collider_center"][2])
    self.write_and_print(file, temp_str)
    temp_str = indent + "CS %f %f %f"
    temp_str %= (object["collider_size"][0],object["collider_size"][1],object["collider_size"][2])
    self.write_and_print(file, temp_str)
self.write_and_print(file, indent + "END")
self.write_and_print(file, '')
```

ファイル出力処理に、今回のカスタムプロパティ分を追加。

`["collider"]`がある場合のみ、`["collider_center"]`と`["collider_size"]`も出力する。

※サンプルプログラム1行分が横に伸びすぎないように無理やりな書き方をしている。
自分で整理しよう

ファイル出力に反映する

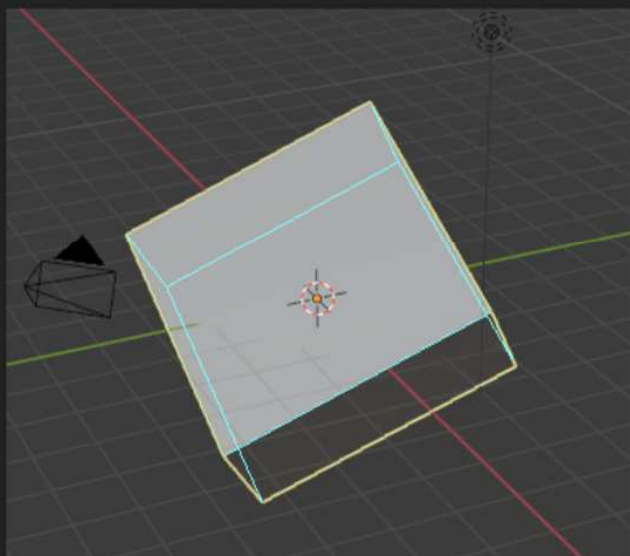
日本工学院専門学校
デザインカレッジ

コライダー関連プロパティを出力するレベルデータファイルに
反映させる。

ファイル出力

実行結果

日本工学院専門学校
デザインカレッジ



コライダーとして設定した各プロパティ、
オブジェクトのトランスフォームが
反映されるようになった。

これでBoxコライダーは完成。

座標に反映

日本工学院専門学校
デザインカレッジ

```
class DrawCollider -draw_collider
```

```
#Boxの8頂点分回す
```

```
for offset in offsets:
```

```
#オブジェクトの中心座標をコピー
```

```
① pos = copy.copy(center) 変更
```

```
#中心点を基準に各頂点ごとにずらす
```

```
pos[0]+=offset[0]*size[0]
```

```
pos[1]+=offset[1]*size[1]
```

```
pos[2]+=offset[2]*size[2]
```

```
#ローカル座標からワールド座標に変換
```

```
+ ② pos = object.matrix_world @ pos
```

```
#頂点データリストに座標を追加
```

```
vertices['pos'].append(pos)
```

Boxの8頂点文、頂点座標を登録する処理の
ところを改造する。

① `object.location` の代わりに
コライダーの中心点を使う。

この時、ワールド座標系でなく、
オブジェクトのローカル座標系に
置き換わっていることに注意。

② オブジェクトのワールド行列を掛けることで、
ワールド座標系に変換する。
(ローカル→ワールド座標変換)
掛け算の演算子は@ (アットマーク)。
このときにオブジェクトのスケール、回転、
平行移動が適用される。

プロパティ取得

日本工学院専門学校
デザインカレッジ

つづき

```
#コライダープロパティがなければ、描画をスキップ
if not "collider" in object:
    continue
```

```
#中心点、サイズの変数を宣言
```

```
center = mathutils.Vector((0,0,0))
size = mathutils.Vector((2,2,2))
```

```
#プロパティから値を取得
```

```
center[0]=object["collider_center"][0]
center[1]=object["collider_center"][1]
center[2]=object["collider_center"][2]
size[0]=object["collider_size"][0]
size[1]=object["collider_size"][1]
size[2]=object["collider_size"][2]
```

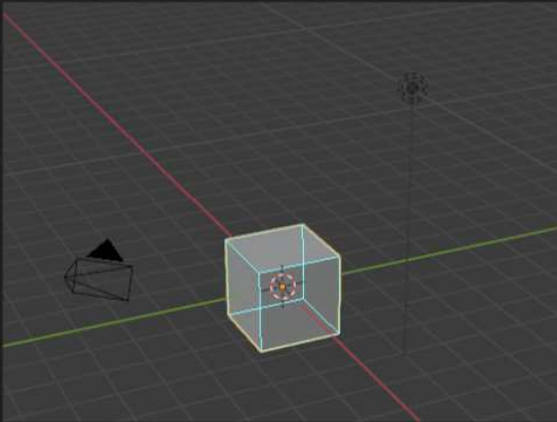
```
#追加前の頂点数
```

```
start = len(vertices["pos"])
```

コライダーがあることが確定している状態で、コライダー用のプロパティに設定した数値を取得する。

実行結果

日本工学院専門学校
デザインカレッジ



カメラやライトにも表示されていた
ボックスが表示されなくなった。

※カメラやライトにコライダーを追加することも
できる。念のため。

描画スキップ

日本工学院専門学校
デザインカレッジ

```
class DrawCollider -draw_collider
```

```
#現在シーンのオブジェクトリストを走査
```

```
for object in bpy.context.scene.objects:
```

```
    #コライダープロパティがなければ、描画をスキップ
```

```
    if not "collider" in object:
```

```
        continue
```

```
    つづく
```

全オブジェクトについて頂点情報の収集をしているfor文で、コライダーのカスタムプロパティがないオブジェクトについて、continue でとばす。

これによって、コライダーがあるオブジェクトのみ描画対象になる。

描画に反映する

カスタムプロパティで

- 中心点
- 大きさ

を持たせるようにしたので、これを3Dビューの描画に反映する。

描画関数は [[資料01_09](#)] で作ったので、それを改造する。

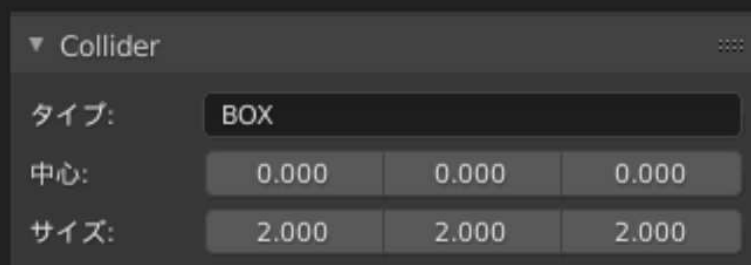
- タイプ

が "BOX" になっていることには今のところ意味はないが、
"collider"プロパティ があるかどうかで、
そのオブジェクトにコライダーを描画するかどうかを決める。

描画に反映する

実行結果

日本工学院専門学校
デザインカレッジ



できた！
しかもラベルテキストを英語で書いたのに
なんかいい感じに翻訳されている。

ちなみにこの時カスタムプロパティを
見るとこんなことになっている。→



クラスリストに登録

日本工学院専門学校
デザインカレッジ

自分で考えよう

`register()` で登録するクラスリストに
パネルを追加しておく。

今までにやった通りにやればO.K.

Panelクラス

日本工学院専門学校
デザインカレッジ

level_editor.py -class OBJECT_PT_collider

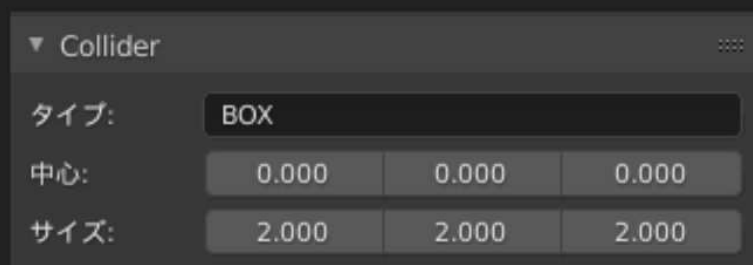
```
#パネル コライダー
class OBJECT_PT_collider(bpy.types.Panel):
    bl_idname = "OBJECT_PT_collider"
    bl_label = "Collider"
    bl_space_type = "PROPERTIES"
    bl_region_type = "WINDOW"
    bl_context = "object"

    # サブメニューの描画
    def draw(self, context):

        #パネルに項目を追加
        if "collider" in context.object:
            #既にプロパティがあれば、プロパティを表示
            self.layout.prop(context.object, '["collider"]', text="Type")
            self.layout.prop(context.object, '["collider_center"]', text="Center")
            self.layout.prop(context.object, '["collider_size"]', text="Size")
        else:
            #プロパティがなければ、プロパティ追加ボタンを表示
            自分で考えよう
```

コライダー設定Panel

日本工学院専門学校
デザインカレッジ



[01_08]で

ファイル名の設定Panelを作ったのと同様に、

コライダー設定Panelを作る。

Unityっぽい配置をイメージしてレイアウトする。

(といってもVectorは自動的にX,Y,Zを横に並べてくれる)

コライダー設定Panel

お試し登録

日本工学院専門学校
デザインカレッジ

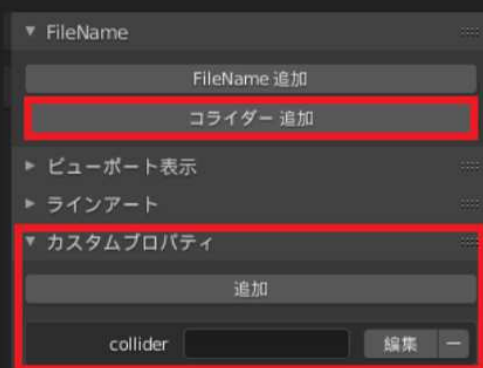
```
# サブメニューの描画  
def draw(self, context):
```

...省略...

```
level_editor.py -class OBJECT_PT_file_name
```

```
+ self.layout.operator(MYADDON_OT_add_collider.bl_idname)
```

Operator が機能しているか確認する為に、ファイル名設定用のPanel (OBJECT_PT_file_name) に登録してみる。



※成功を確認したら、上記コードは消すこと。

クラスリストに登録

日本工学院専門学校
デザインカレッジ

```
# Blenderに登録するクラスリスト
classes = (
    MYADDON_OT_stretch_vertex,
    MYADDON_OT_create_ico_sphere,
    MYADDON_OT_export_scene,
    TOPBAR_MT_my_menu,
    MYADDON_OT_add_filename,
    OBJECT_PT_file_name,
    MYADDON_OT_add_collider,
)
```

register() で登録するクラスリストに
オペレータを追加しておく。

**登録順は特に関係ないので、
自分で整理しなおしてもよい。**

カスタムプロパティ追加Operator

日本工学院専門学校
デザインカレッジ

```
level_editor.py -class MYADDON_OT_add_collider
```

```
#オペレータ カスタムプロパティ['collider']追加
class MYADDON_OT_add_collider(bpy.types.Operator):
    bl_idname = "myaddon.myaddon_ot_add_collider"
    bl_label = "コライダー 追加"
    bl_description = "['collider']カスタムプロパティを追加します"
    bl_options = {"REGISTER", "UNDO"}

    def execute(self, context):

        #['collider']カスタムプロパティを追加
        context.object["collider"] = "BOX"
        context.object["collider_center"] = mathutils.Vector((0,0,0))
        context.object["collider_size"] = mathutils.Vector((2,2,2))

        return {"FINISHED"}
```

追加ボタンはOperatorをMenuやPanelに登録することで実現できるので、カスタムプロパティ追加Operatorを作成する。

今回はコライダー用に

- collider
- collider_center
- collider_size

の3つのプロパティを追加する。

モジュールのインポート

日本工学院専門学校
デザインカレッジ

```
import mathutils
```

スクリプトの先頭で、Blender Python API の mathutils モジュールをインポートしておく。

mathutils の詳細は [リファレンス](#) を確認。

オイラー角(Euler)、行列(Matrix)、クォータニオン(Quaternion)、ベクトル(Vector)、カラー(Color)のクラスを提供してくれる。

サブモジュールにはさらに色々な数学の便利機能がある。

今回は Vector の型を使いたいので、インポートする。

コライダー追加ボタン

日本工学院専門学校
デザインカレッジ



コライダー 追加

[01_08]で

ファイル名のカスタムプロパティを追加
ボタンを作ったのと同様に、

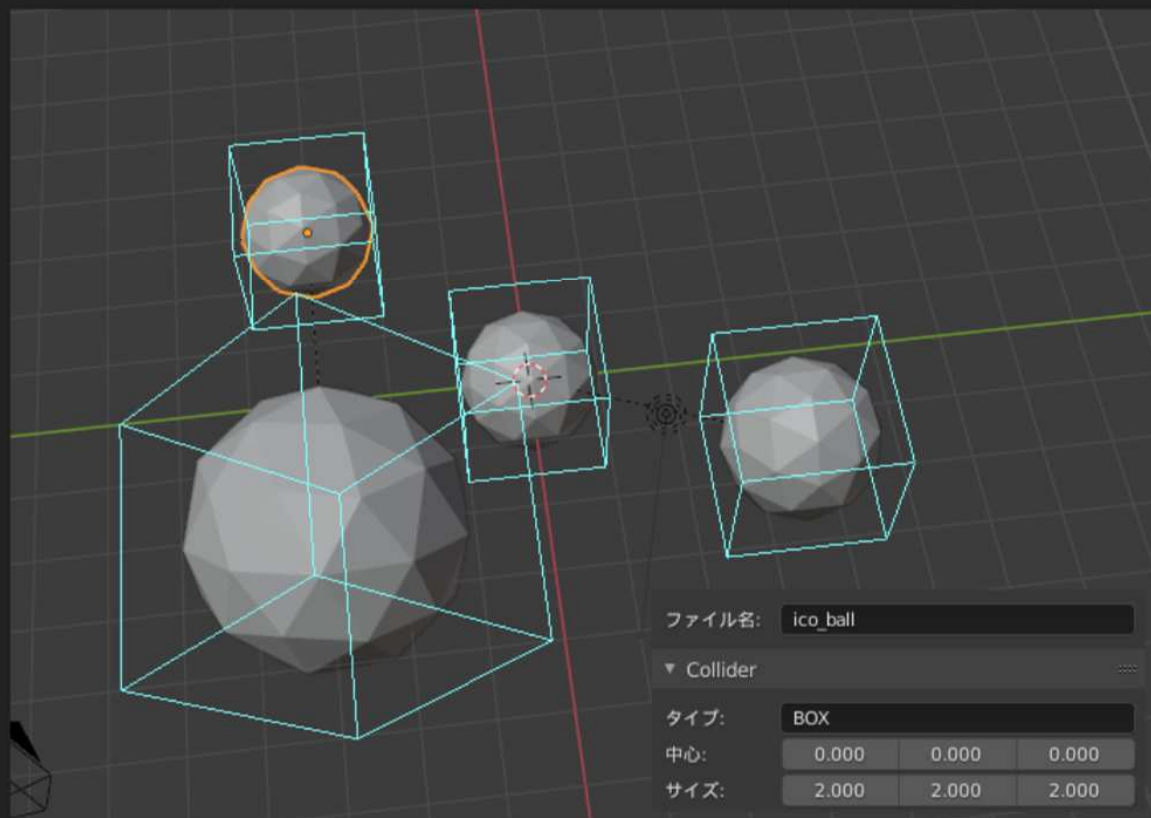
コライダー追加ボタンを作る。

カスタムプロパティ自体はBlenderの機能なので、
「プロパティ追加ボタン」でなくても同じ機能は実現できるが、
利便性を上げる為に作る。

コライダー追加ボタン

Boxコライダー

日本工学院専門学校
デザインカレッジ



UnityをイメージしたBoxコライダーを
Blenderから配置・調整できるように
する。

これができれば、レベルデータとして
設定したいあらゆる項目を
Blenderから設定可能になるだろう。
(もちろん公式リファレンスを調べながら)

Boxコライダー

今回の目標

日本工学院専門学校
デザインカレッジ

- Boxコライダーのメニュー・描画を実装する
（レベルエディタの機能拡張）
- Boxコライダーのデータをファイル出力に加える

TL1

01_10.Boxコライダー

日本工学院専門学校
デザインカレッジ